

Exp 9

Write the python program to implement A* algorithm.

Aim:

To write a python program to implement A* algorithm.

Algorithm:

Step1 Initialize the open list with the starting node and the closed list as an empty set.

Step 2: Initialize the g score of the starting node to be 0 and the f score of the starting node to be the heuristic estimate of the distance to the goal.

Step 3: While the open list is not empty, select the node with the lowest f score and set it as

the current node.

Step 4: If the current node is the goal node, return the reconstructed path.

Remove the current node from the open list and add it to the closed list.

Step 5: For each neighbor of the current node, calculate the tentative g score as the g score of the current node plus the distance between the current node and the neighbor.

Step 6: If the neighbor is already in the closed list, skip it.

Step 7: If the neighbor is not in the open list, add it and set its g score and f score.

Step 8 :If the neighbor is in the open list but the tentative g score is greater than or equal to its current g score, skip it.

Step 9 : Otherwise, update the came from dictionary, g score, and f score of the neighbor.

Step 10 : Repeat steps 3-10 until the open list is empty.

If the goal node was never reached, return failure.

Program:

```
from queue import PriorityQueue
```

```
# Graph with cost
```

```
graph = {  
    'A': [('B', 1), ('C', 3)],  
    'B': [('D', 3), ('E', 6)],  
    'C': [('F', 5)],  
    'D': [],  
    'E': [('F', 2)],  
    'F': []  
}
```

Heuristic values

```
heuristic = {  
    'A': 7,  
    'B': 6,  
    'C': 2,  
    'D': 1,  
    'E': 1,  
    'F': 0  
}
```

```
def a_star(start, goal):  
    pq = PriorityQueue()  
    pq.put((0, start))  
    cost = {start: 0}  
    parent = {start: None}  
    while not pq.empty():  
        current = pq.get()[1]  
        if current == goal:  
            break  
        for neighbor, weight in graph[current]:  
            new_cost = cost[current] + weight
```

```

        if neighbor not in cost or new_cost < cost[neighbor]:
            cost[neighbor] = new_cost
            priority = new_cost + heuristic[neighbor]
            pq.put((priority, neighbor))
            parent[neighbor] = current

# Find path
path = []
node = goal
while node is not None:
    path.append(node)
    node = parent[node]
path.reverse()
print("Shortest Path:", path)
print("Total Cost:", cost[goal])

# Driver code
a_star('A', 'F')

```

Output:

```

---
>>>
Shortest Path: ['A', 'C', 'F']
Total Cost: 8
>>> |

```

Result:

Thus, the Program was executed and the output was found to be Correct.